Техническая спецификация и системная архитектура платформы DevCrew (Версия 2.0)

Created with NotebookLM • 10/31/2026

---

exported: 2026-10-31T15:16:44.278Z source: NotebookLM type: report title: "Техническая спецификация и системная архитектура платформы DevCrew (Версия 2.0)"

Техническая спецификация и системная архитектура платформы DevCrew (Версия 2.0)

Exported at: 10/31/2026, 4:16:44 PM

Техническая спецификация и системная архитектура платформы DevCrew (Версия 2.0)

Платформа DevCrew представляет собой специализированный программный комплекс для оркестрации автономной инженерной команды искусственного интеллекта, интегрированной с репозиториями исходного кода GitHub. Система автоматизирует полный жизненный цикл инженерной задачи — от анализа входящего issue и декомпозиции архитектурных требований до модификации программного кода в изолированной ветке, сквозного автоматизированного код-ревью, структурной верификации и формирования pull request. В настоящей расширенной версии технической спецификации подробно представлены реляционная модель данных (полная схема всех таблиц Drizzle ORM с типами полей, связями и ограничениями), глубокий анализ интеграции внешних сервисов с разбором архитектуры подключения инструментов и сопоставлением с протоколом MCP (Model Context Protocol), а также сквозная матрица межсервисного взаимодействия.

---

1. Архитектурный манифест и целеполагание проекта

1.1. Проблематика разработки программного обеспечения и предпосылки автоматизации

В классических процессах сопровождения и развития программного обеспечения значительная доля рабочего времени квалифицированных инженеров расходуется на рутинные и повторяющиеся операции:

- Первичный разбор, категоризацию, воспроизведение и оценку входящих отчетов об ошибках (bug reports) и запросов на улучшение (feature requests).
- Локализацию проблемных областей в кодовой базе и написание шаблонного связующего кода (boilerplate) для типовых доработок интерфейса или бэкенда.
- Многоэтапное согласование pull request, проверку соблюдения внутренних стандартов оформления кода, актуализацию базовых веток и рутинный контроль сборки.
- Разрешение тривиальных конфликтов слияния и ручную синхронизацию веток разработки.

Привлечение разработчиков-людей к подобным операциям сопряжено с высокой себестоимостью человеко-часов, потерями контекста при постоянных переключениях между тикетами и неравномерной пропускной способностью инженерной команды. В то же время прямое применение изолированных больших языковых моделей без строгого оркестратора не решает системную проблему:

- Отсутствует замкнутый контур исполнения: разработчик вынужден вручную переносить сгенерированный код, тестировать его и самостоятельно публиковать коммиты.
- Отсутствует изоляция и ролевой контроль: универсальная модель без жестких ограничений может повредить конфигурационные файлы (`.env` , `.github/workflows`) или внести несовместимые правки в ветку по умолчанию (`main`).
- Отсутствует гарантированное бюджетное управление: неконтролируемые вызовы API могут привести к нежелательному росту затрат на токены.

1.2. Миссия и ключевые задачи DevCrew

Проект DevCrew решает указанные вызовы путем формирования виртуального автономного инженерного отдела, состоящего из четырех специализированных ИИ-агентов: **Analyst (Аналитик)**, **Developer (Разработчик)**, **Reviewer (Ревьюер)** и **Tester (Тестировщик)**.

Система обеспечивает реализацию следующих ключевых задач:

1. **Сквозная автономность инженерного цикла**: Преобразование тикетов GitHub Issues в готовые, проверенные и протестированные pull request без необходимости ручного написания кода инженером.
2. **Строгая изоляция и сохранность репозитория**: Исключение прямого доступа агентов к защищенным веткам (default branch), файлам непрерывной интеграции (`.github/workflows`) и переменным окружения (`.env`), а также безопасное хранение ключей доступа с шифрованием на уровне строк базы данных.
3. **Аппаратное бюджетное управление**: Непрерывный мониторинг операционных расходов на обращение к языковым моделям и вызовы инструментов с немедленным отсечением задач, превышающих лимиты затрат или максимальное число итераций.
4. **Аудит надежности и рентабельности агентов**: Интеграция со специализированными платформами статического анализа и оценки устойчивости ИИ (Switch Trust API) для валидации системных инструкций и контроля надежности.
5. **Прозрачная визуализация операционной деятельности**: Предоставление интерактивной канбан-доски и виртуального рабочего пространства офиса (Live Agent Office), отображающих активность, текущий статус и вычислительные затраты каждого агента в реальном времени.

1.3. Сравнительный анализ парадигм разработки

В представленной ниже таблице сопоставлены ключевые операционные характеристики традиционной инженерной разработки, использования персональных ИИ-ассистентов и платформы DevCrew:

Параметр оценки	Традиционная разработка (Human Team)	Изолированные AI-ассистенты (Copilot / Chat)	Платформа DevCrew
Инициация задачи	Ручной триаж тимлидом или проектным менеджером	Ручной ввод промпта инженером в IDE	Автоподхват issues из GitHub или запуск в один клик
Разделение обязанностей	Раздельные роли (аналитик, программист, QA, лид)	Одна модель выполняет все задачи в одном общем контексте	Четыре специализированных субагента с выделенными промптами и инструментами
Создание веток и фиксация кода	Ручное создание веток и коммитов человеком	Копирование сгенерированных фрагментов кода вручную	Автоматическое ветвление от default branch, атомарные коммиты через GitHub REST API
Контроль качества (Code Review)	Человеческое ревью с ожиданием в часы или дни	Отсутствует, полная ответственность на инженере	Автономное ревью (Reviewer) и верификация файлов (Tester) с циклом доработок
Финансовый контроль	Фиксированная заработная плата сотрудников	Фиксированная подписка без проектного учета затрат	Гранулярные лимиты в центах на каждого агента с валидацией перед каждым шагом
Фоновое исполнение	Ограничено рабочим графиком специалистов	Блокирующий синхронный интерфейс в окне редактора	Асинхронные очереди фоновых задач (Cloudhike Jobs) с подписанными вебхуками

---

2. Структура проекта, технологии и организация кодовой базы

2.1. Спецификация технологического стека

Архитектура DevCrew спроектирована на основе современного веб-стека, оптимизированного для server-side рендеринга, безопасного управления сессиями и распределенной асинхронной обработки задач:

- **Фреймворк приложения**: Next.js 16 на базе React 19 с применением архитектуры App Router, React Server Components (RSC) и Server Actions.
- **Язык программирования**: TypeScript в строгом режиме типизации (strict mode).
- **Стилизация и дизайн-система**: Tailwind CSS v4 с централизованным управлением глобальными цветовыми токенами в файле `app/globals.css`.
- **Компонентная база пользовательского интерфейса**: Библиотека shadcn/ui (base-nova поверх `@base-ui/react`), векторные пиктограммы `lucide-react`, drag-and-drop механизм на базе библиотеки `@dnd-kit`.
- **Аутентификация и сессии**: Облачный сервис Better Auth (хостинг на Cloudhike Auth service) с поддержкой входа по Email/Password, Google OAuth и GitHub OAuth.
- **База данных и объектно-реляционное отображение**: Управляемая СУБД PostgreSQL 14+, взаимодействующая с приложением через Drizzle ORM с декларативной схемой в `db/schema.ts`.
- **Объектное хранилище файлов**: S3-совместимое частное хранилище (Cloudhike Storage) для безопасной загрузки логотипов и медиа-активов рабочих пространств.
- **Интеграция с системой контроля версий**: GitHub REST API через специализированное приложение GitHub OAuth App с шифрованием токенов на уровне строк таблицы (`TOKEN_ENCRYPTION_KEY`).
- **Исполнительный контур языковых моделей**: OpenAI Responses API с официальным SDK `openai`.
- **Система управления фоновыми задачами**: Cloudhike Jobs API для оркестрации распределенных асинхронных заданий с проверкой подписи `x-cloudhike-signature` (в локальной среде разработки предусмотрен fallback на системный механизм `next/after`).
- **Аудит надежности и рентабельности агентов**: Switch Trust API и Switch ROI для оценки устойчивости конфигураций и мониторинга финансовой эффективности.

2.2. Модульная структура репозитория

Кодовая база структурирована по функциональным слоям с четким разделением клиентского представления, серверных действий и низкоуровневых сервисных библиотек:

- `app/`: Маршрутизация приложения. Включает изолированные группы маршрутов авторизации `(auth)`, маршруты рабочих пространств `workspaces/`, панель управления `dashboard/` (включая подразделы `board`, `ai-team`, `pull-requests`, `activity`, `settings`), системные эндпоинты `api/` (обработчики GitHub OAuth, Cloudhike Jobs, Better Auth и Switch Trust), а также глобальную таблицу стилей `globals.css`.
- `components/`: Презентационный слой интерфейса, сгруппированный по предметным областям: компоненты канбан-доски (`board/`), карточки конфигурации агентов (`ai-team/`), интерактивное рабочее пространство офиса (`office/`), аналитические сводки (`overview/`), интерфейсы слияния pull request (`pull-requests/`) и базовые примитивы UI (`ui/`).
- `db/`: Слой данных. Содержит файл `schema.ts`, описывающий реляционные сущности (`users`, `workspaces`, `tasks`, `agents`, `agent_activities`, `task_runs`, `pull_requests`, `switch_connections`), а также модуль инициализации клиента Drizzle `index.ts`.
- `lib/`: Серверная бизнес-логика. Модули управления аутентификацией (`auth.ts`), рабочими пространствами (`workspaces.ts`), задачами (`tasks.ts`, `board.ts`), агентами (`agents.ts`, `agent-defaults.ts`), конвейером исполнения (`task-runs.ts`), инструментами манипуляции файлами (`agent-tools.ts`), контролем затрат (`agent-budget.ts`), интеграцией с GitHub (`github.ts`, `github-repo.ts`, `github-issues.ts`), криптографией (`crypto.ts`), клиентом фоновых задач (`jobs.ts`) и интеграцией со Switch Trust (`switch-trust.ts`, `switch-scan.ts`).
- `office/`: Исходные JSON-пакеты сцены AgentWorld (`office_1.json`), определяющие пространственные координаты столов, аватары и спрайты анимаций.
- `public/`: Статические клиентские ресурсы, скомпилированные спрайты персонажей `public/office/` и фоновое изображение `public/modern-office.png`.
- `scripts/`: Инженерные скрипты сборки, включая `extract-office.mjs` для конвертации base64-спрайтов из JSON в оптимизированные графические файлы.
- `proxy.ts`: Пограничный middleware-прокси для оптимистичной валидации сессий и перенаправления неавторизованных пользователей.

2.3. Сценарии автоматизации и команды сборки

В таблице ниже приведены ключевые скрипты управления жизненным циклом проекта, зарегистрированные в `package.json`:

Команда	Функциональное назначение
npm run dev	Запуск локального сервера разработки Next.js на порту 3000
npm run build	Компиляция и оптимизация production-сборки приложения
npm run start	Запуск скомпилированного production-сервера
npx tsc --noEmit	Статическая проверка типов TypeScript без генерации JS-файлов
npm run db:push	Прямая синхронизация декларативной схемы db/schema.ts с базой данных Postgres
npm run db:generate	Генерация миграционных SQL-файлов в каталог drizzle/
npm run db:studio	Запуск веб-интерфейса Drizzle Studio для визуального просмотра таблиц БД
npm run office:extract	Запуск скрипта extract-office.mjs для парсинга office/office_1.json в спрайты и layout

---

3. Реляционная модель данных: спецификация схемы Drizzle ORM

Слой хранения данных DevCrew опирается на реляционную СУБД PostgreSQL под управлением Drizzle ORM. Архитектура схемы спроектирована с учетом строгой изоляции данных по рабочим пространствам (multi-tenancy через `workspaceId`), поддержки асинхронных состояний фонового конвейера и прозрачного финансового учета.

3.1. Архитектурная декомпозиция реляционных таблиц

База данных содержит восемь ключевых сущностей:

1. `users`: Профили учетных записей пользователей. Создаются автоматически при первом успешном входе через сервис Better Auth (без необходимости вебхуков).
2. `workspaces`: Рабочие пространства проекта. Каждое пространство закреплено за владельцем (`userId`) и связывает проект с ровно одним GitHub-репозиторием, а также хранит ссылки на кастомный логотип в S3 и настройки автоматического выполнения задач.
3. `tasks`: Задачи канбан-доски. Отражают как задачи, синхронизированные из GitHub Issues, так и пользовательские тикеты, отслеживая их статус прохождения по стадиям конвейера (`todo`, `in_development`, `in_review`, `testing`, `done`).
4. `agents`: Персонализированные конфигурации ИИ-агентов для конкретного воркспейса. Хранят специализированные системные промпты, описания, списки активных навыков, лимиты максимального числа итераций (`maxTurns`), установленный бюджет (`budgetCents`) и накопленный расход средств (`spentCents`).
5. `agent\_activities`: Неизменяемый журнал аудита действий агентов. Содержит детальные записи о каждом вызове инструментов (`read_file`, `write_file`, `submit_*`), стоимости операции и времени исполнения.
6. `task\_runs`: Экземпляры выполнения конвейера по конкретной задаче. Отслеживают текущий активный этап (`stage`), выходные спецификации Аналитика, рабочую ветку Разработчика, количество раундов доработки (`reworkRounds`) и замечания Ревьюера.
7. `pull\_requests`: Сведения о pull request, сгенерированных агентом Tester в целевом репозитории GitHub, включая URL, номер PR, ветку, статус слияния и время закрытия.
8. `switch\_connections`: Параметры интеграции рабочего пространства с платформой аудита Switch Trust. Хранят идентификатор воркспейса Switch и зашифрованный API-ключ.

3.2. Спецификация полей и структуры таблиц

В представленной ниже сводной таблице подробно описаны все поля, их типы данных и назначение в реляционной структуре проекта:

Таблица	Поле	Тип данных	Ограничения и связи	Назначение
users	id	text	Primary Key	Идентификатор пользователя из сервиса Better Auth
users	name, email	text	Not Null, Unique (email)	Имя пользователя и контактный адрес электронной почты
users	image	text	Nullable	Аватар учетной записи пользователя
users	created_at, updated_at	timestamp	With Timezone, Default Now	Метки времени создания и обновления профиля
workspaces	id	uuid	Primary Key, Default Random	Уникальный идентификатор рабочего пространства
workspaces	user_id	text	FK -> users.id (Cascade)	Владелец рабочего пространства
workspaces	name	text	Not Null	Название рабочего пространства
workspaces	icon, logo_url	text	Nullable	Предустановленная пиктограмма или ссылка на S3-логотип
workspaces	github_repo	text	Not Null	Привязанный репозиторий в формате owner/repo
workspaces	encrypted_github_token	text	Not Null	Зашифрованный токен доступа GitHub OAuth App
workspaces	auto_pickup_tasks	boolean	Not Null, Default false	Флаг автозапуска входящих задач агентом Analyst
workspaces	auto_merge_pr	boolean	Not Null, Default false	Флаг автоматического слияния одобренных PR
tasks	id	uuid	Primary Key, Default Random	Уникальный идентификатор задачи
tasks	workspace_id	uuid	FK -> workspaces.id (Cascade)	Принадлежность к рабочему пространству
tasks	github_issue_number	integer	Nullable	Номер тикета в GitHub Issues (при наличии)
tasks	title, description	text	Not Null (title), Nullable	Наименование и детальное описание требований
tasks	type, priority	text	Default feature, medium	Тип (issue/bug/feature) и приоритет задачи
tasks	status	text	Default todo	Статус канбан-доски (todo/in_dev/in_review/testing/done)
tasks	assigned_agent	text	Nullable	Активный агент, выполняющий задачу
tasks	pr_url	text	Nullable	URL созданного pull request
agents	id	uuid	Primary Key, Default Random	Идентификатор конфигурации агента

Таблица	Поле	Тип данных	Ограничения и связи	Назначение
agents	workspace_id	uuid	FK -> workspaces.id (Cascade)	Принадлежность к рабочему пространству
agents	role, name	text	Not Null	Роль (analyst/developer/reviewer/tester) и имя
agents	instructions	text	Not Null	Ролевой системный промпт агента
agents	skills	json	Array of strings	Список активных функциональных навыков
agents	budget_cents	integer	Default 5000 (50$)	Установленный лимит затрат на модель в центах
agents	spent_cents	integer	Default 0	Фактически израсходованная сумма в центах
agents	max_turns	integer	Not Null (15 - 60)	Максимальное число шагов агента на одну задачу
agents	status	text	Default idle	Текущий статус (idle/running/over_budget)
agents	trust_score	integer	Nullable (0-100)	Индекс надежности по оценке Switch Trust
agent_activities	id	uuid	Primary Key, Default Random	Идентификатор записи действия
agent_activities	workspace_id, task_id	uuid	FK -> workspaces.id, tasks.id	Привязка действия к пространству и задаче
agent_activities	agent_role, action_type	text	Not Null	Роль агента и имя инструмента (read/write/submit)
agent_activities	details	text	JSON string	Параметры вызова инструмента
agent_activities	cost_cents	integer	Default 0	Стоимость шага в центах
task_runs	id	uuid	Primary Key, Default Random	Идентификатор прогона задачи
task_runs	task_id, workspace_id	uuid	FK -> tasks.id, workspaces.id	Связь с задачей и рабочим пространством
task_runs	stage	text	Not Null	Текущая активная стадия конвейера
task_runs	analyst_output	text	Nullable	План и спецификация Аналитика
task_runs	dev_branch	text	Nullable	Имя рабочей ветки Разработчика в GitHub
task_runs	rework_feedback	text	Nullable	Замечания Ревьюера при отправке на доработку
task_runs	rework_rounds	integer	Default 0	Количество завершенных циклов доработки
pull_requests	id	uuid	Primary Key, Default Random	Идентификатор записи pull request

Таблица	Поле	Тип данных	Ограничения и связи	Назначение
pull_requests	task_id, workspace_id	uuid	FK -> tasks.id, workspaces.id	Связь с задачей и пространством
pull_requests	github_pr_number	integer	Not Null	Номер pull request в репозитории GitHub
pull_requests	pr_url, branch	text	Not Null	Веб-адрес pull request и имя исходной ветки
pull_requests	status	text	Default open	Статус PR (open/merged/closed)
switch_connections	id	uuid	Primary Key, Default Random	Идентификатор интеграции
switch_connections	workspace_id	uuid	Unique, FK -> workspaces.id	Изоляция подключения на уровне воркспейса
switch_connections	encrypted_api_key	text	Not Null	Зашифрованный API-ключ платформы Switch AI
switch_connections	tenant_id	text	Nullable	Идентификатор рабочей области в Switch Trust

3.3. Исходный код схемы базы данных (`db/schema.ts`)

Ниже представлен декларативный код описания всех сущностей базы данных на TypeScript с использованием библиотеки Drizzle ORM:

```
import { pgTable, text, timestamp, integer, boolean, uuid, json } from "drizzle-orm/pg-core";
import { relations } from "drizzle-orm";

export const users = pgTable("users", {
  id: text("id").primaryKey(),
  name: text("name").notNull(),
  email: text("email").notNull().unique(),
  image: text("image"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const workspaces = pgTable("workspaces", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: text("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  name: text("name").notNull(),
  icon: text("icon"),
  logoUrl: text("logo_url"),
  githubRepo: text("github_repo").notNull(),
  githubInstallationId: text("github_installation_id"),
  encryptedGithubToken: text("encrypted_github_token").notNull(),
  autoPickupTasks: boolean("auto_pickup_tasks").default(false).notNull(),
  autoMergePr: boolean("auto_merge_pr").default(false).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const tasks = pgTable("tasks", {
  id: uuid("id").defaultRandom().primaryKey(),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete: "cascade" }),
  githubIssueNumber: integer("github_issue_number"),
  title: text("title").notNull(),
  description: text("description"),
  type: text("type").notNull().default("feature"),
  priority: text("priority").notNull().default("medium"),
  status: text("status").notNull().default("todo"),
  assignedAgent: text("assigned_agent"),
  estimatedPoints: integer("estimated_points"),
  dueDate: timestamp("due_date", { withTimezone: true }),
  prUrl: text("pr_url"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const agents = pgTable("agents", {
  id: uuid("id").defaultRandom().primaryKey(),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete: "cascade" }),
  role: text("role").notNull(),
  name: text("name").notNull(),
  description: text("description"),
  instructions: text("instructions").notNull(),
  skills: json("skills").$type<string[]>().default([]).notNull(),
  budgetCents: integer("budget_cents").default(5000).notNull(),
  spentCents: integer("spent_cents").default(0).notNull(),
  maxTurns: integer("max_turns").notNull(),
  status: text("status").default("idle").notNull(),
  trustScore: integer("trust_score"),
  lastScannedAt: timestamp("last_scanned_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
```

```
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const agentActivities = pgTable("agent_activities", {
  id: uuid("id").defaultRandom().primaryKey(),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete: "cascade" }),
  agentRole: text("agent_role").notNull(),
  taskId: uuid("task_id").references(() => tasks.id, { onDelete: "set null" }),
  actionType: text("action_type").notNull(),
  details: text("details"),
  costCents: integer("cost_cents").default(0).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
});

export const taskRuns = pgTable("task_runs", {
  id: uuid("id").defaultRandom().primaryKey(),
  taskId: uuid("task_id").notNull().references(() => tasks.id, { onDelete: "cascade" }),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete: "cascade" }),
  stage: text("stage").notNull(),
  analystOutput: text("analyst_output"),
  devBranch: text("dev_branch"),
  reworkFeedback: text("rework_feedback"),
  reworkRounds: integer("rework_rounds").default(0).notNull(),
  startedAt: timestamp("started_at", { withTimezone: true }).defaultNow().notNull(),
  finishedAt: timestamp("finished_at", { withTimezone: true }),
});

export const pullRequests = pgTable("pull_requests", {
  id: uuid("id").defaultRandom().primaryKey(),
  taskId: uuid("task_id").notNull().references(() => tasks.id, { onDelete: "cascade" }),
  workspaceId: uuid("workspace_id").notNull().references(() => workspaces.id, { onDelete: "cascade" }),
  githubPrNumber: integer("github_pr_number").notNull(),
  prUrl: text("pr_url").notNull(),
  title: text("title").notNull(),
  branch: text("branch").notNull(),
  status: text("status").default("open").notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  mergedAt: timestamp("merged_at", { withTimezone: true }),
});

export const switchConnections = pgTable("switch_connections", {
  id: uuid("id").defaultRandom().primaryKey(),
  workspaceId: uuid("workspace_id").notNull().unique().references(() => workspaces.id, { onDelete: "cascade" }),
  encryptedApiKey: text("encrypted_api_key").notNull(),
  tenantId: text("tenant_id"),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});
```

---

4. Архитектура интеграции внешних сервисов и протокол MCP

Платформа DevCrew построена вокруг концепции специализированных облачных интеграций. Каждый сторонний сервис решает конкретную инженерную задачу, формируя устойчивую инфраструктурную экосистему.

4.1. Каталог внешних сервисов, решаемые проблемы и функциональные возможности

В таблице ниже приведен детальный перечень сервисов, используемых в архитектуре DevCrew:

Внешний сервис	Решаемая инженерная проблема	Ключевые функциональные возможности
Cloudhike (cloudhike.dev)	Устранение фрагментации бэкенда (All-in-One Cloud Backend)	Управляемый Postgres 14+, размещенная аутентификация Better Auth, приватное S3-хранилище, распределенные очереди фоновых задач Cloudhike Jobs с подписанными вебхуками
GitHub (REST API & OAuth App)	Прямая интеграция с кодовой базой и процессами CI/CD	Чтение issues, автосинхронизация канбан-доски, создание веток от default branch, атомарная запись файлов, сравнение веток (diffs), открытие и слияние PR
OpenAI Platform (API)	Исполнительный вычислительный контур для рассуждений агентов	Предоставление моделей семейства gpt-4o, нативный вызов функций (Tool Calling), расчет токенов и списание затрат в центах
Switch AI (switchagents.ai)	Независимый аудит надежности и учет эффективности затрат	Статическое тестирование промптов агентов (Switch Trust), оценка устойчивости, мониторинг окупаемости и затрат (Switch ROI), GitHub Action для CI/CD
ngrok (Tunneling Service)	Доставка облачных вебхуков на локальную машину разработчика	Создание защищенного публичного HTTPS-туннеля к localhost:3000 для приема обратных вызовов от Cloudhike Jobs и Switch Trust

4.2. Архитектурный анализ Model Context Protocol (MCP)

При проектировании мультиагентных систем актуален вопрос применения стандарта **Model Context Protocol (MCP)**, предложенного для унификации взаимодействия языковых моделей с внешними источниками данных и инструментами через выделенные JSON-RPC процессы.

В архитектуре DevCrew разделены два контура взаимодействия с инструментами:

1. **Продакшен-рантайм конвейера (In-App Runtime Tooling)**: В работающем приложении агенты взаимодействуют с файловой системой и репозиторием **без использования внешнего демона MCP-сервера**. Инструменты реализованы нативно через механизм **OpenAI Tool Calling** в серверном модуле `lib/agent-tools.ts`. Декларации инструментов передаются в модель в виде JSON-схем, а вызовы диспетчеризуются напрямую серверным кодом Next.js через клиент `@octokit/rest`. *Обоснование решения*: Прямой вызов TypeScript-функций исключает накладные расходы на сетевой транспорт и межпроцессный обмен, снижает время отклика и позволяет атомарно синхронизировать вызовы инструментов с транзакциями базы данных Postgres и списанием бюджетов агентов.
2. **Контур разработки кодовой базы (Development-time Tooling)**: В ходе создания проекта автор использовал ИИ-агента парного программирования **Claude Code** внутри VS Code. В этом сценарии агент разработки подключался к локальному рабочему окружению, считывая архитектурный манифест `AGENTS.md` и исполняя консольные команды управления зависимостями и миграциями базы данных.

4.3. Примеры программного подключения внешних сервисов

Ниже представлены рабочие примеры серверных модулей подключения внешних сервисов в кодовой базе проекта.

4.3.1. Клиент очередей фоновых задач Cloudhike Jobs (`lib/jobs.ts`)

```
import crypto from "crypto";

interface EnqueueJobParams {
  endpoint: string;
  payload: Record<string, any>;
}

export async function enqueueCloudhikeJob({ endpoint, payload }: EnqueueJobParams) {
  const jobsApiUrl = process.env.JOBS_API_URL;
  const jobsApiKey = process.env.JOBS_API_KEY;
  const appUrl = process.env.NEXT_PUBLIC_APP_URL;

  if (!jobsApiUrl || !jobsApiKey) {
    console.log(`[Jobs Local Fallback] Асинхронный запуск задачи на ${endpoint}`);
    return { id: "local-job-" + Date.now(), status: "queued" };
  }

  const targetUrl = `${appUrl}${endpoint}`;

  const response = await fetch(`${jobsApiUrl}/runs`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-api-key": jobsApiKey,
    },
    body: JSON.stringify({
      target_url: targetUrl,
      payload,
      max_attempts: 3,
    }),
  });

  if (!response.ok) {
    const errorText = await response.text();
    throw new Error(`Ошибка постановки задачи в Cloudhike Jobs: ${response.status} ${errorText}`);
  }

  return await response.json();
}
```

4.3.2. Исполнительный клиент OpenAI Responses API (`lib/openai.ts`)

```
import OpenAI from "openai";

if (!process.env.OPENAI_API_KEY) {
  throw new Error("Переменная окружения OPENAI_API_KEY не задана в конфигурации .env");
}

export const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});
```

4.3.3. Фабрика клиента GitHub REST API (`lib/github.ts`)

```
import { Octokit } from "@octokit/rest";
import { decryptToken } from "@/lib/crypto";

export async function createGitHubClientForWorkspace(encryptedToken: string): Promise<Octokit> {
  const encryptionKey = process.env.TOKEN_ENCRYPTION_KEY;
  if (!encryptionKey) {
    throw new Error("TOKEN_ENCRYPTION_KEY отсутствует в конфигурации окружения.");
  }

  const plainAccessToken = decryptToken(encryptedToken, encryptionKey);

  return new Octokit({
    auth: plainAccessToken,
  });
}
```

4.3.4. Клиент статического аудита Switch Trust API (`lib/switch-trust.ts`)

```
export async function triggerSwitchTrustScan(params: {
  apiKey: string;
  agentId: string;
  agentRole: string;
  instructions: string;
}) {
  const response = await fetch("/api/switch/evaluations", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "x-switch-api-key": params.apiKey,
    },
    body: JSON.stringify({
      agent_id: params.agentId,
      agent_name: params.agentRole,
      evaluation_type: "agent_code_scan_static",
      system_instructions: params.instructions,
    }),
  });

  return await response.json();
}
```

---

5. Межсервисное взаимодействие и сквозной жизненный цикл данных

Архитектура DevCrew обеспечивает непрерывную циркуляцию данных между всеми подключенными облачными платформами. Взаимодействие выстроено по событийно-ориентированной модели, гарантирующей согласованность состояний репозитория, базы данных и интерфейса.

5.1. Матрица межсервисного взаимодействия

В таблице ниже систематизированы каналы коммуникации между внешними сервисами, протоколы обмена и решаемые задачи:

Инициирующий сервис	**Принимающий сервис**	**Передаваемые данные и протокол**	**Инженерная цель взаимодействия**
Cloudhike Better Auth	Cloudhike Postgres	Профиль пользователя, email, id через внутренний коннектор	Создание записи пользователя в таблице users при первом входе

Инициирующий сервис	Принимающий сервис	Передаваемые данные и протокол	Инженерная цель взаимодействия
GitHub OAuth App	Next.js API & Postgres	Временный code обменивается на Access Token (зашифрован в DB)	Получение и безопасное сохранение прав доступа к репозиторию
Next.js Server Action	Cloudhike Jobs	POST /runs с подписанным эндпоинтом назначения (target_url)	Асинхронный запуск долгоиграющей стадии конвейера (/api/jobs/task-run)
Cloudhike Jobs	Next.js Webhook API	POST /api/jobs/task-run с проверкой подписи x-cloudhike-signature	Доставка триггера выполнения шага конвейера на сервер приложения
Оркестратор (Next.js)	OpenAI Responses API	Контекст тикета, системный промпт, JSON-схема доступных инструментов	Интеллектуальный анализ, генерация кода и принятие решений агентом
OpenAI Responses API	Cloudhike Postgres	Расход токенов (prompt/completion) → списание spentCents	Немедленная фиксация затрат и аппаратный контроль бюджетов агентов
Субагенты (Next.js)	GitHub REST API	REST-запросы Octokit (ветки, создание файлов, получение diffs, PR)	Автономное выполнение инженерных изменений в репозитории проекта
Next.js Server Action	Switch Trust API	Инструкции агента, инструменты, конфигурация через REST API	Статический аудит на корректность инструкций и безопасность
Switch Trust	Next.js Webhook API	POST /api/switch/agents/[id] с HMAC-токеном верификации	Передача итогового Trust Score (0-100) и сохранение в таблице agents
Next.js Client	Cloudhike S3 Storage	Multipart upload приватного логотипа рабочего пространства	Хранение брендированных медиа-активов воркспейса
ngrok	Next.js Local Server	Проброс внешних HTTPS-запросов Cloudhike Jobs/Switch Trust на порт 3000	Тестирование полного цикла распределенной архитектуры на локальной машине

5.2. Пошаговая хронология сквозного исполнения задачи

Сквозной сценарий обработки тикета демонстрирует синергию интегрированных сервисов:

1. **Аутентификация**: Пользователь открывает страницу входа и выбирает вход через GitHub Sign-In. Облачный сервис **Cloudhike Better Auth** проверяет учетные данные и инициирует создание записи в таблице `users` в **Cloudhike Postgres**.
2. **Привязка репозитория**: Пользователь создает воркспейс, загружает логотип в **Cloudhike S3 Storage** и авторизует доступ к GitHub через **GitHub OAuth App**. Сервер приложения шифрует полученный токен алгоритмом AES-256 с помощью ключа `TOKEN_ENCRYPTION_KEY` и сохраняет его в таблице `workspaces`.
3. **Синхронизация тикетов**: Модуль `lib/github-issues.ts` запрашивает открытые issues из GitHub API с помощью Octokit и автоматически сохраняет их в таблицу `tasks` со статусом `todo`.
4. **Инициация конвейера**: Пользователь нажимает кнопку **Run** на карточке задачи. Сервер Next.js проверяет лимит затрат агента в таблице `agents` и отправляет запрос на создание задачи в **Cloudhike Jobs** (`POST /runs`).
5. **Шаг Аналитика**: Воркер Cloudhike Jobs вызывает вебхук `/api/jobs/task-run`. Оркестратор поднимает системный промпт Аналитика, отправляет запрос в **OpenAI API**, фиксирует расход токенов в `agents.spentCents` и сохраняет план работ в `task_runs.analystOutput`. Оркестратор ставит в Cloudhike Jobs следующую задачу — для Разработчика.
6. **Шаг Разработчика**: Разработчик запрашивает создание ветки через **GitHub REST API**, последовательно вызывает инструменты записи файлов (`write_file`), вносит изменения в кодовую базу и закрывает шаг вызовом `submit_development`. Затраты списываются через `recordAgentSpend()`.
7. **Шаг Ревьюера**: Ревьюер запрашивает дифф изменений через `octokit.rest.repos.compareCommits`. Если код соответствует стандартам, он вызывает `submit_reviewer`. При наличии дефектов тикет отправляется обратно Разработчику на доработку (rework loop).
8. **Шаг Тестировщика и PR**: Тестировщик верифицирует изменения и открывает Pull Request через **GitHub REST API**, записывая метаданные в таблицу `pull_requests`. Карточка на канбан-доске переходит в статус завершения с отображением ссылки на созданный PR.

9. **Аудит безопасности**: В параллельном контуре пользователь может нажать **Scan** на карточке агента. Сервер обращается к **Switch Trust API**, инициируя статический аудит промпта и инструментов. По завершении оценки Switch Trust присылает обратный вебхук, и вычисленный `trustScore` обновляется в базе данных.

---

6. Архитектура мультиагентного конвейера и алгоритмы исполнения

В основе DevCrew лежит строго детерминированный конвейер, управляемый конечным автоматом. Жизненный цикл задачи распределен между узкоспециализированными агентами, каждый из которых обладает собственным набором системных правил и прав доступа.

6.1. Оркестратор пошагового цикла (Turn Loop)

Каждый агент исполняется в изолированном цикле, ограниченном параметром `maxTurns` (по умолчанию: Analyst — 20, Developer — 60, Reviewer — 20, Tester — 15). На последней итерации система директивно принуждает модель вызвать инструмент передачи управления следующему этапу.

```
 import { openai } from "@/lib/openai";
import { assertAgentCanRun, recordAgentSpend } from "@/lib/agent-budget";
import { executeAgentTool, getAgentToolsDefinition } from "@/lib/agent-tools";
import { db } from "@/db";
import { tasks, taskRuns, agentActivities } from "@/db/schema";
import { eq } from "drizzle-orm";

interface AgentExecutionParams {
  taskId: string;
  runId: string;
  agentRole: "analyst" | "developer" | "reviewer" | "tester";
  systemPrompt: string;
  taskContext: string;
  maxTurns: number;
  workspaceId: string;
  repoDetails: { owner: string; repo: string; branch: string; token: string };
}

export async function executeAgentTurnLoop(params: AgentExecutionParams) {
  const { taskId, runId, agentRole, systemPrompt, taskContext, maxTurns, workspaceId, repoDetails } = params;

  const conversationMessages: any[] = [
    { role: "system", content: systemPrompt },
    { role: "user", content: taskContext }
  ];

  let currentTurn = 0;
  let stageCompleted = false;

  while (currentTurn < maxTurns && !stageCompleted) {
    currentTurn++;

    // 1. Аппаратная проверка финансового лимита агента перед каждым шагом
    await assertAgentCanRun(workspaceId, agentRole);

    // 2. Получение декларации доступных инструментов для данной роли
    const isLastTurn = currentTurn === maxTurns;
    const tools = getAgentToolsDefinition(agentRole, { forceSubmitOnLastTurn: isLastTurn });

    // 3. Обращение к OpenAI Responses API
    const response = await openai.chat.completions.create({
      model: "gpt-4o",
      messages: conversationMessages,
      tools: tools.length > 0 ? tools : undefined,
      tool_choice: isLastTurn ? { type: "function", function: { name: `submit_${agentRole}` } } : "auto"
    });

    const choice = response.choices[0];
    const message = choice.message;
    conversationMessages.push(message);

    // 4. Фиксация фактического расхода токенов и списание средств из бюджета
    if (response.usage) {
      await recordAgentSpend({
        workspaceId,
        agentRole,
        promptTokens: response.usage.prompt_tokens,
        completionTokens: response.usage.completion_tokens,
        model: "gpt-4o"
      });
```

```
    }

    // 5. Обработка вызова инструментов
    if (message.tool_calls && message.tool_calls.length > 0) {
      for (const toolCall of message.tool_calls) {
        const functionName = toolCall.function.name;
        const functionArgs = JSON.parse(toolCall.function.arguments);

        if (functionName === `submit_${agentRole}`) {
          stageCompleted = true;
          await handleStageHandoff(taskId, runId, agentRole, functionArgs);
        }

        const executionResult = await executeAgentTool({
          toolName: functionName,
          args: functionArgs,
          repoDetails,
          agentRole
        });

        await db.insert(agentActivities).values({
          workspaceId,
          agentRole,
          taskId,
          actionType: functionName,
          details: JSON.stringify(functionArgs),
          createdAt: new Date()
        });

        conversationMessages.push({
          role: "tool",
          tool_call_id: toolCall.id,
          content: JSON.stringify(executionResult)
        });
      }
    } else {
      if (isLastTurn) {
        stageCompleted = true;
      }
    }
  }

  if (!stageCompleted) {
    throw new Error(`Агент ${agentRole} исчерпал лимит итераций (${maxTurns}) без формального завершения этапа.`);
  }
}
```

6.2. Диспетчеризация переходов между субагентами (`handleStageHandoff`)

Сердцем оркестратора является функция межстадийного перехода, обновляющая статус задачи на канбан-доске и регистрирующая следующее фоновое задание в очереди Cloudhike Jobs:

```
 import { db } from "@/db";
import { tasks, taskRuns } from "@/db/schema";
import { eq } from "drizzle-orm";
import { enqueueCloudhikeJob } from "@/lib/jobs";

export async function handleStageHandoff(
  taskId: string,
  runId: string,
  currentStage: "analyst" | "developer" | "reviewer" | "tester",
  outputData: any
) {
  switch (currentStage) {
    case "analyst":
      await db.update(tasks).set({ status: "in_development" }).where(eq(tasks.id, taskId));
      await db.update(taskRuns).set({ stage: "developer", analystOutput: outputData.plan }).where(eq(taskRuns.id, runId));
      await enqueueCloudhikeJob({ endpoint: "/api/jobs/task-run", payload: { taskId, runId, nextStage: "developer" } });
      break;

    case "developer":
      await db.update(tasks).set({ status: "in_review" }).where(eq(tasks.id, taskId));
      await db.update(taskRuns).set({ stage: "reviewer", devBranch: outputData.branch }).where(eq(taskRuns.id, runId));
      await enqueueCloudhikeJob({ endpoint: "/api/jobs/task-run", payload: { taskId, runId, nextStage: "reviewer" } });
      break;

    case "reviewer":
      if (outputData.approved) {
        await db.update(tasks).set({ status: "testing" }).where(eq(tasks.id, taskId));
        await db.update(taskRuns).set({ stage: "tester" }).where(eq(taskRuns.id, runId));
        await enqueueCloudhikeJob({ endpoint: "/api/jobs/task-run", payload: { taskId, runId, nextStage: "tester" } });
      } else {
        await db.update(tasks).set({ status: "in_development" }).where(eq(tasks.id, taskId));
        await db.update(taskRuns).set({ stage: "developer", reworkFeedback: outputData.feedback }).where(eq(taskRuns.id,
runId));
        await enqueueCloudhikeJob({ endpoint: "/api/jobs/task-run", payload: { taskId, runId, nextStage: "developer" } });
      }
      break;

    case "tester":
      await db.update(tasks).set({ status: "done", prUrl: outputData.prUrl }).where(eq(tasks.id, taskId));
      await db.update(taskRuns).set({ stage: "completed", finishedAt: new Date() }).where(eq(taskRuns.id, runId));
      break;
  }
}
```

6.3. Контроль бюджета и финансовых лимитов (`lib/agent-budget.ts`)

Безопасность затрат обеспечивается двухуровневой защитой: предварительной блокировкой запуска задачи при превышении лимита и атомарной фиксацией стоимости каждого вызова API.

```
 import { db } from "@/db";
import { agents } from "@/db/schema";
import { eq, and } from "drizzle-orm";

export async function assertAgentCanRun(workspaceId: string, agentRole: string): Promise<void> {
  const [agentRecord] = await db
    .select()
    .from(agents)
    .where(and(eq(agents.workspaceId, workspaceId), eq(agents.role, agentRole)));

  if (!agentRecord) {
    throw new Error(`Конфигурация агента с ролью ${agentRole} не найдена в рабочем пространстве.`);
  }

  if (agentRecord.spentCents >= agentRecord.budgetCents) {
    throw new Error(
      `Агент ${agentRole} заблокирован: исчерпан установленный бюджет ` +
      `($${agentRecord.spentCents / 100}$ из $${agentRecord.budgetCents / 100}$).`
    );
  }
}

export async function recordAgentSpend(params: {
  workspaceId: string;
  agentRole: string;
  promptTokens: number;
  completionTokens: number;
  model: string;
}): Promise<void> {
  const { workspaceId, agentRole, promptTokens, completionTokens, model } = params;

  const costCents = calculateTokenCostCents(model, promptTokens, completionTokens);

  const [current] = await db
    .select({ spentCents: agents.spentCents })
    .from(agents)
    .where(and(eq(agents.workspaceId, workspaceId), eq(agents.role, agentRole)));

  if (current) {
    await db
      .update(agents)
      .set({ spentCents: current.spentCents + costCents })
      .where(and(eq(agents.workspaceId, workspaceId), eq(agents.role, agentRole)));
  }
}

function calculateTokenCostCents(model: string, inputTokens: number, outputTokens: number): number {
  const inputRatePerMillion = 250;
  const outputRatePerMillion = 1000;
  const cost = (inputTokens / 1_000_000) * inputRatePerMillion + (outputTokens / 1_000_000) * outputRatePerMillion;
  return Math.max(1, Math.round(cost));
}
```

6.4. Набор инструментов агента и защита репозитория (`lib/agent-tools.ts`)

Агенты взаимодействуют с GitHub репозиторием через строго определенные серверные функции. Файловые инструменты содержат встроенные проверки безопасности:

```
import { Octokit } from "@octokit/rest";

export async function executeAgentTool(params: {
  toolName: string;
  args: any;
  repoDetails: { owner: string; repo: string; branch: string; token: string };
  agentRole: string;
}) {
  const { toolName, args, repoDetails, agentRole } = params;
  const octokit = new Octokit({ auth: repoDetails.token });

  if (toolName === "write_file" || toolName === "delete_file") {
    const targetPath = args.path as string;

    if (
      targetPath.startsWith(".github/workflows") ||
      targetPath.startsWith(".env") ||
      targetPath === ".gitignore"
    ) {
      throw new Error(`Отказ в доступе: модификация пути ${targetPath} категорически запрещена правилами безопасности DevCrew.`);
    }

    if (repoDetails.branch === "main" || repoDetails.branch === "master") {
      throw new Error("Отказ в доступе: прямая запись в ветку по умолчанию запрещена. Изменения допустимы только в выделенной рабочей ветке.");
    }
  }

  switch (toolName) {
    case "read_file":
      const fileData = await octokit.rest.repos.getContent({
        owner: repoDetails.owner,
        repo: repoDetails.repo,
        path: args.path,
        ref: repoDetails.branch
      });
      return { content: Buffer.from((fileData.data as any).content, "base64").toString("utf-8") };

    case "write_file":
      let sha: string | undefined;
      try {
        const existing = await octokit.rest.repos.getContent({
          owner: repoDetails.owner,
          repo: repoDetails.repo,
          path: args.path,
          ref: repoDetails.branch
        });
        sha = (existing.data as any).sha;
      } catch (err) {
        sha = undefined;
      }

      await octokit.rest.repos.createOrUpdateFileContents({
        owner: repoDetails.owner,
        repo: repoDetails.repo,
        path: args.path,
        message: `[DevCrew ${agentRole}] Обновление файла ${args.path}`,
        content: Buffer.from(args.content).toString("base64"),
        branch: repoDetails.branch,
```

```
        sha
      });
      return { success: true, path: args.path };

    case "get_diff":
      const comparison = await octokit.rest.repos.compareCommits({
        owner: repoDetails.owner,
        repo: repoDetails.repo,
        base: "main",
        head: repoDetails.branch
      });
      return {
        filesChanged: comparison.data.files?.map(f => ({ filename: f.filename, status: f.status, patch: f.patch }))
      };

    default:
      throw new Error(`Неизвестный инструмент: ${toolName}`);
  }
}
```

---

7. Формализованные спецификации системных промптов агентов

Каждый агент платформы обладает детерминированной ролевой моделью, зафиксированной в системном промпте. Промпты исключают произвольные рассуждения и ориентируют модель на строгое следование протоколам взаимодействия.

7.1. Системный промпт Аналитика (Analyst Agent)

```
Вы — ведущий системный аналитик автономной команды разработки DevCrew.
Ваша миссия: проанализировать входящую задачу (Issue, Bug или Feature Request) из репозитория GitHub, сопоставить ее с существующей архитектурой проекта и составить исчерпывающий, структурированный план реализации для Разработчика (Developer).

ВАШИ ОБЯЗАННОСТИ:
1. Тщательно изучите заголовок, описание и контекст переданного тикета.
2. Используйте инструменты чтения файловой системы репозитория для исследования структуры проекта, используемых библиотек и зависимостей.
3. Локализуйте конкретные модули, компоненты и файлы, которые подлежат созданию или модификации.
4. Сформируйте однозначные критерии приемки (Acceptance Criteria) и опишите возможные граничные случаи (edge cases).
5. Передайте управление Разработчику с помощью вызова инструмента `submit_analyst`.

ПРАВИЛА И ОГРАНИЧЕНИЯ:
- Вы работаете исключительно в режиме чтения. Вам категорически запрещено модифицировать или создавать файлы в репозитории.
- Ваш план должен быть конкретным: указывайте точные имена файлов, структуры экспортируемых функций и алгоритмические шаги.
- Вы должны уложиться в отведенный лимит итераций (не более 20 шагов). На последнем шаге вы обязаны вызвать `submit_analyst`.
```

7.2. Системный промпт Разработчика (Developer Agent)

```
Вы — старший инженер-программист автономной команды разработки DevCrew.
Ваша миссия: реализовать архитектурный план и требования, подготовленные Аналитиком, путем создания качественного, типизированного и полностью рабочего программного кода в выделенной рабочей ветке репозитория GitHub.

ВАШИ ОБЯЗАННОСТИ:
1. Создайте и зафиксируйте изменения в изолированной ветке, ответвленной от основной ветки репозитория.
2. Используйте инструменты `read_file`, `write_file` и `delete_file` для последовательной реализации функционала.
3. Пишите чистый код на TypeScript, строго соблюдая стандарты оформления проекта, принятые соглашения об именовании и правила дизайн-системы (Tailwind CSS токены).
4. Убедитесь в отсутствии синтаксических ошибок, циклических зависимостей и недопустимых импортов.
5. После завершения всех правок вызовите инструмент `submit_development`, приложив краткую сводку внесенных изменений.

ПРАВИЛА И ОГРАНИЧЕНИЯ:
- Категорически запрещено вносить изменения в защищенные файлы: любые файлы переменных окружения (.env*), файлы рабочих процессов GitHub Actions (.github/workflows/*) и конфигурации репозитория.
- Никогда не отправляйте коммиты напрямую в ветки main или master.
- Ваши изменения должны быть атомарными и логически сгруппированными.
- Вы должны уложиться в отведенный лимит шагов (максимум 60 итераций). На последнем шаге обязательно вызывается `submit_development`.
```

7.3. Системный промпт Ревьюера (Reviewer Agent)

```
Вы — ведущий инженер по качеству кода и архитектурному ревью команды DevCrew.
Ваша миссия: провести независимую и бескомпромиссную инспекцию диффа изменений, подготовленного Разработчиком, сопоставив его с исходными требованиями Аналитика.

ВАШИ ОБЯЗАННОСТИ:
1. Запросите и внимательно изучите diff между рабочей веткой и базовой веткой репозитория с помощью инструмента `get_diff`.
2. Оцените код по следующим критериям:
   - Полнота реализации всех пунктов плана Аналитика.
   - Соответствие типов TypeScript и отсутствие неявных приведений (`any`).
   - Безопасность (отсутствие уязвимостей, утечек секретов, небезопасных вызовов).
   - Чистота архитектуры, читаемость и соблюдение стилистики проекта.
3. Примите решение:
   - В случае обнаружения проблем верните задачу на доработку Разработчику с аргументированным списком правок через `request_rework`.
   - При полном соответствии вызовите инструмент `submit_reviewer` для передачи задачи Тестировщику.

ПРАВИЛА И ОГРАНИЧЕНИЯ:
- Вы не имеете права вносить изменения в код самостоятельно.
- Ваши замечания должны быть конструктивными, со ссылками на конкретные строки диффа и предложениями по исправлению.
- Вы должны завершить инспекцию в пределах 20 итераций.
```

7.4. Системный промпт Тестировщика (Tester Agent)

```
Вы — инженер по верификации и выпуску релизов автономной команды DevCrew.
Ваша миссия: провести финальную структурную проверку подготовленных артефактов, сформировать pull request в репозитории GitHub и обеспечить корректное сопровождение жизненного цикла задачи.

ВАШИ ОБЯЗАННОСТИ:
1. Проверьте состояние рабочей ветки, факт наличия коммитов и целостность файловой структуры.
2. Убедитесь, что все затронутые файлы находятся в правильных директориях и проект готов к открытию pull request.
3. Сформируйте информативный Pull Request на GitHub:
   - Заголовок должен четко отражать суть задачи (формат: `feat(scope): ...` или `fix(scope): ...`).
   - В теле PR подробно опишите: реализованный функционал, план Аналитика, подтверждение Ревьюера и результаты финальной верификации.
4. Зафиксируйте данные созданного PR в базе данных системы через инструмент `submit_tester`.
5. При возникновении конфликтов слияния или падении автоматических проверок CI оформите новый тикет с подробным отчетом об ошибке и добавьте его в колонку Todo.

ПРАВИЛА И ОГРАНИЧЕНИЯ:
- Тестировщик отвечает за открытие PR и финальный перевод карточки задачи в статус завершения конвейера.
- Лимит шагов исполнения — 15 итераций. На финальном шаге вызывается `submit_tester`.
```

---

8. Инженерная хроника разработки системы (Prompt Ledger)

Платформа DevCrew была построена инкрементальным путем с помощью ИИ-агента парного программирования Claude Code. Процесс разработки состоял из 18 последовательных инженерных промптов, каждый из которых добавлял законченный функциональный модуль и поддерживал актуальность архитектурного манифеста `AGENTS.md` .

Сводный реестр всех этапов эволюции кодовой базы:

№	Дата	Функциональный модуль	Архитектурное назначение и ключевые артефакты
1	22 окт	Аутентификация (Authentication)	Развертывание Better Auth (Cloudhike), таблицы users, сессионные куки
2	22 окт	Рабочие пространства и GitHub OAuth	Создание воркспейсов, шифрование токенов (TOKEN_ENCRYPTION_KEY), манифест AGENTS.md
3	22 окт	Каркас Dashboard и навигация	Разметка панели управления, сворачиваемый сайдбар, палитра токенов globals.css
4	22 окт	Макет страницы Overview	Карточки открытых задач и PR, плейсхолдер виртуального офиса, недавний лог
5	22 окт	Корректировка авторизации	Устранение потери токенов при редиректах доменов хостинга Cloudhike (отключение www)
6	22 окт	Канбан-доска (@dnd-kit)	5 вертикальных колонок (Todo..Done), перетаскивание карточек, диалог создания задач
7	22 окт	Конфигурация команды AI Team	Генерация 4 агентов по умолчанию, модальные окна настройки навыков и бюджетов
8	22 окт	Аппаратный контроль бюджета	Модуль lib/agent-budget.ts, функции assertAgentCanRun() и recordAgentSpend()
9	22 окт	Синхронизация с GitHub Issues	Автозагрузка issues на доску, кнопка ручного запуска Run на карточках
10	23 окт	Планирование Switch Trust	Предварительное исследование спецификаций Switch API без написания кода

№	Дата	Функциональный модуль	Архитектурное назначение и ключевые артефакты
11	23 окт	Интеграция Switch Trust и Jobs	Статический аудит агентов, очереди фоновых задач Cloudhike Jobs (/api/jobs/switch-scan)
12	23 окт	Оптимизация интерфейса Switch	Удаление дублирующих метрик, вывод круговых диаграмм оценки надежности
13	23 окт	Конвейер агентов (AI Task Pipeline)	Модуль lib/task-runs.ts, пошаговый turn loop, создание веток и публикация PR
14	23 окт	Реальные данные Overview/Activity	Замена моков реальными метриками запусков, расходами агентов и таймлайном
15	24 окт	Интерактивный Live Agent Office	Пиксельная визуализация рабочих столов и статусов агентов в реальном времени
16	25 окт	Оптимизация графики офиса	Скрипт extract-office.mjs, конвертация base64 спрайтов сцены в оптимизированные PNG
17	25 окт	Публичный Landing Page	Финализация посадочной страницы и удаление неиспользуемых плейсхолдеров
18	25 окт	Передача кодовой базы (Handoff)	Создание документации разработчика: README.md, .env.example и prompt.md

Ключевые инженерные выводы при создании мультиагентных систем:

1. **Декларативное описание пользовательских сценариев**: Формулировка промптов в терминах пользовательских потоков (*«Когда пользователь нажимает X, открыть диалог Y, передающий данные в Z»*) обеспечивает значительно более высокую связность кода по сравнению с перечислением разрозненных компонентов.
2. **Фаза планирования перед интеграцией сторонних API**: Выделение отдельного промпта с жесткой директивой «Только спланируй, код не пиши» позволяет проверить совместимость внешних сервисов (как в случае со Switch Trust) и избежать генерации неработающих заглушек.
3. **Изоляция правил в проектных манифестах**: Фиксация архитектурных и стилистических ограничений в едином файле `AGENTS.md` гарантирует их непрерывное соблюдение всеми сессиями ИИ-кодинга без необходимости повторения инструкций в каждом новом промпте.
4. **Ранний аппаратный контроль бюджетов**: Внедрение проверок лимитов затрат до запуска реальных пайплайнов полностью нивелирует финансовые риски неконтролируемого зацикливания языковых моделей.